Software Engineering

BCSE301L

Module 3

Dr. Naveenkumar Jayakumar
Dept. of Computational Intelligence
PRP-217-4

Software Requirements

- Descriptions of services a system should provide.
  - Constraints on system operation.
  - Reflect customer needs (e.g., controlling a device, placing an order).

- Software requirements are the descriptions of the services that a software system should provide and the constraints under which it must operate. They essentially define what the software should do to meet the needs of its users and stakeholders.

Software Requirements

- What the software should do: This encompasses the **functionalities and features** expected from the system. Examples include:
  - Processing user inputs
  - Storing and retrieving data
  - Generating reports
  - Controlling external devices

Software Requirements

- Constraints on operation: These are **limitations** or restrictions **on how the software should function**. Examples include:
  - Performance requirements (e.g., response time, throughput)
  - Security requirements (e.g., data encryption, access control)
  - Usability requirements (e.g., ease of use, accessibility)
  - Compatibility requirements (e.g., operating system compatibility)

Software Requirements - Types

Types of software requirements

Business requirements	User requirements	Software requirements
Outline measurable goals for the business.	Reflect specific user needs or expectations.	Identify features, functions, non-functional requirements and use cases.
Define the why behind a software project.	Describe the who of a software project.	Delve into the how of a software project.
Match project goals to stakeholder goals.	Highlight how users interact with it.	Describe software as functional modules and non-functional attributes.
Maintain a BRD with requirements, updates or changes.	Create a URS, or make them part of the BRD.	Compose an SRS, and, optionally, an FRS.

Software Requirements - Types

Business Requirements

- The high-level goals and objectives that a software system must achieve to support the organization's strategic business needs. They focus on the "why" behind the software and its impact on the business.

Functional Requirements

- Describe the specific functions that the software must perform

Non-Functional Requirements

- Describe the quality attributes of the software, such as performance, usability, and security.

User requirements

- Describe the needs and expectations of the end-users of the software

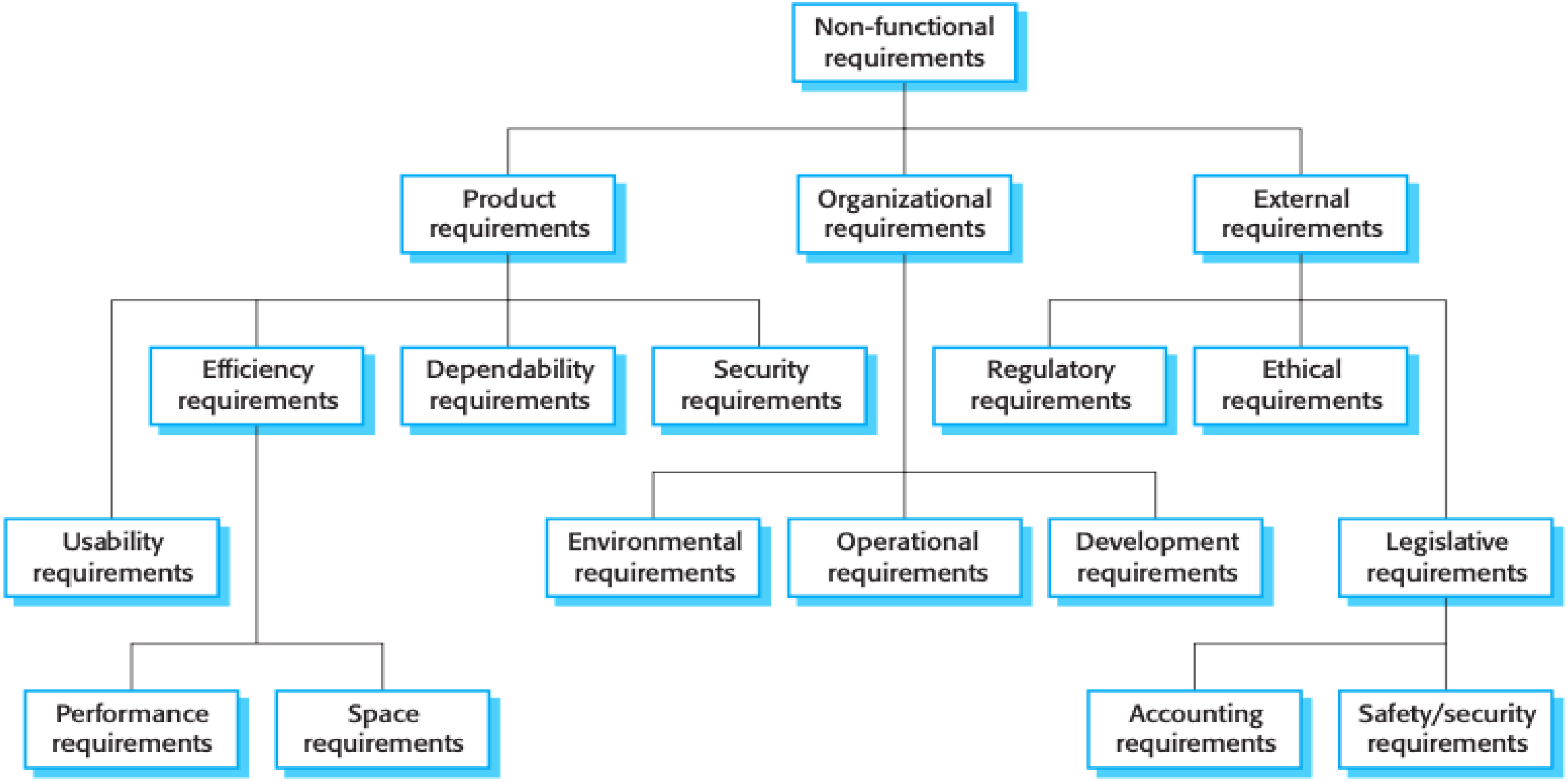
System requirements:

- Describe the overall requirements of the entire system, including the software and hardware components

Software Requirements - Types

	Functional requirements	Non-functional requirements
Objective	Defining what the product does	Describing how the product works
End result	Product features	Product properties
Focus	On user needs	On user expectations
Documentation	Software requirements specification, use cases, user stories, work breakdown structures, prototypes	Software Requirements Specification
Origin	Usually defined by end users	Usually defined by developers or other tech experts
Types	Business rules, administrative functions, data management, transaction management, etc.	Usability, reliability, performance, security, scalability, compatibility, etc.

Software Requirements – Non-Functional



Software Requirements Engineering

- A Process of
  - finding
  - analyzing
  - documenting and
  - checking requirements.

Software Requirements Engineering

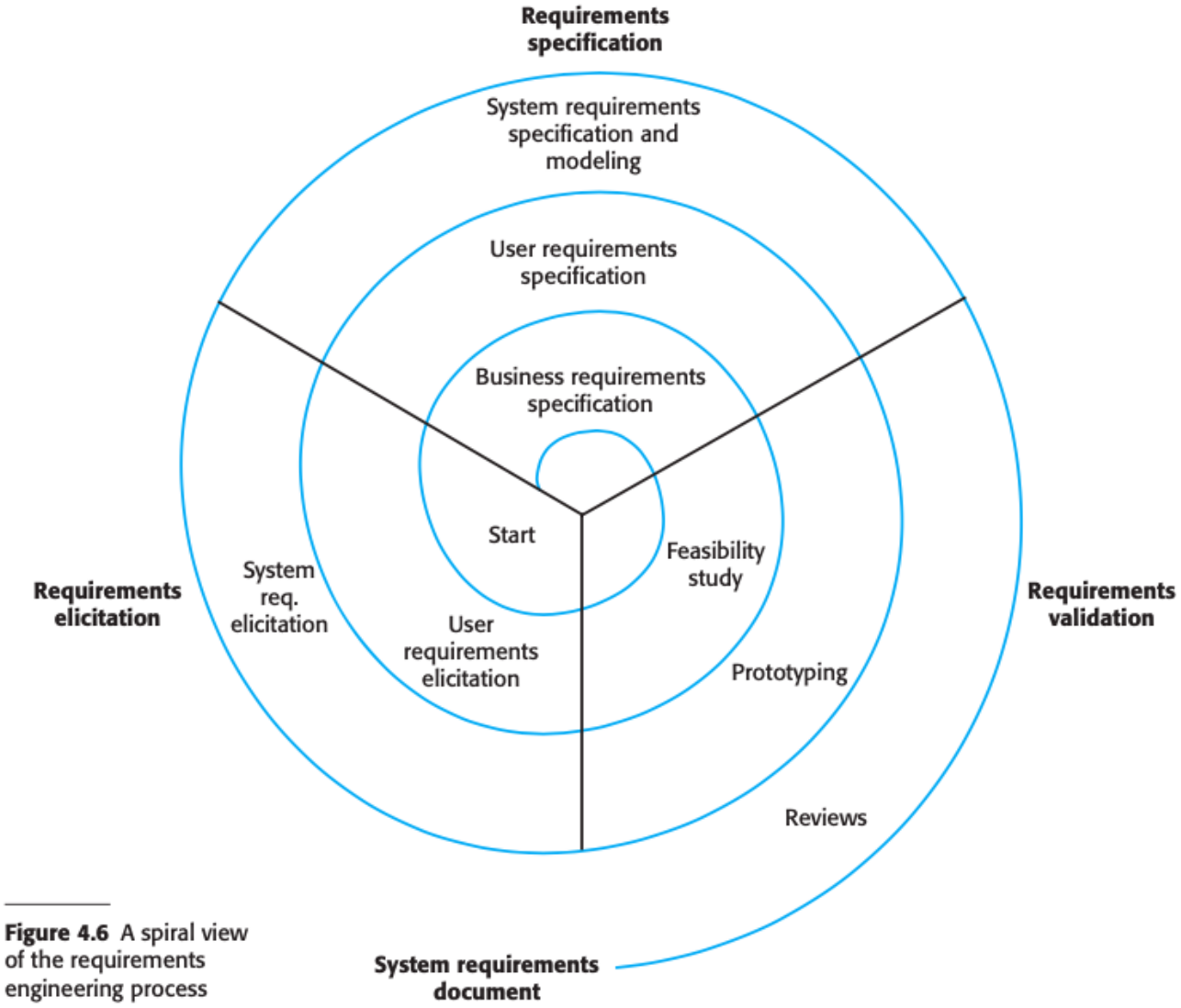


Figure 4.6 A spiral view of the requirements engineering process

Software Requirements Engineering - Activities

- **Feasibility Study:**

- Purpose: To assess the technical, economic, and operational feasibility of the proposed system.

- Activities: Market analysis, technology assessment, risk analysis, cost-benefit analysis.

Software Requirements Engineering - Activities

- **Requirements Elicitation:**

- **Purpose:** To gather information about the needs and expectations of stakeholders (users, customers, domain experts).

- **Techniques**:
  - **Interviews:** One-on-one or group discussions to gather detailed information.
  - **Surveys:** Questionnaires to collect data from a large number of stakeholders.
  - **Observation**: Observing users interacting with existing systems or performing their tasks.
  - **Focus groups**: Group discussions to explore ideas and gather feedback.
  - **Workshops**: Collaborative sessions to brainstorm and refine requirements.
  - **Prototyping**: Creating early versions of the system to gather user feedback.

Software Requirements Engineering - Activities

- **Requirements Analysis:**

- **Purpose:** To analyze the elicited information, identify conflicts, and prioritize requirements.

- **Techniques:**
  - **Use case analysis**: Describing the interactions between users and the system.
  - **Data flow diagrams**: Visualizing the flow of data through the system.
  - **Requirement decomposition**: Breaking down complex requirements into smaller, more manageable ones.
  - **Risk analysis**: Identifying potential risks and developing mitigation strategies.

Software Requirements Engineering - Activities

- **Requirements Specification:**

- **Purpose:** To document the requirements in a clear, concise, and unambiguous manner.

- **Deliverables**: Software Requirements Specification (SRS) document, which includes:
- **Functional requirements**: What the system should do.
- **Non-functional requirements**: Quality attributes of the system (e.g., performance, usability, security).
- **User requirements**: Requirements from the perspective of end-users.
- **System requirements**: Requirements from the perspective of the system developers.

Software Requirements Engineering - Activities

- **Requirements Validation:**

- **Purpose:** To ensure that the requirements are correct, complete, consistent, and feasible.

- **Techniques:**
  - **Reviews and inspections**: Formal or informal reviews by stakeholders and experts.
  - **Prototyping**: Creating prototypes to validate user requirements.
  - **Testing:** Conducting tests to verify that the system meets the specified requirements.

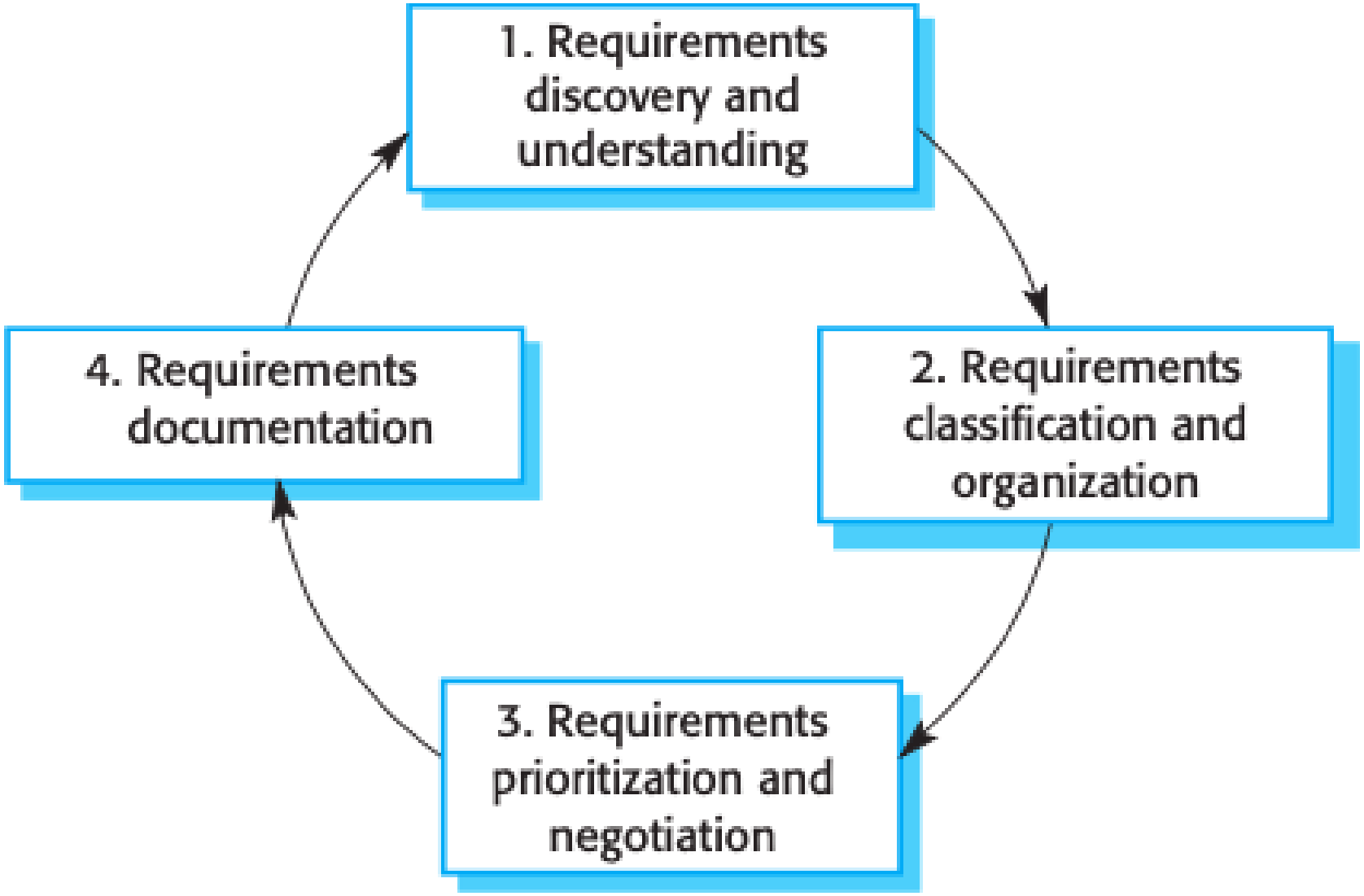
Software Requirements Engineering - Activities

- **Requirements Management:**

- **Purpose:** To track changes to requirements, manage conflicts, and ensure that the system remains aligned with the evolving needs of stakeholders.

- **Activities:**
  - **Change control**: Establishing a process for managing changes to requirements.
  - **Version control**: Maintaining different versions of the requirements documents.
  - **Traceability:** Linking requirements to other project artifacts (e.g., design documents, test cases).

Requirement Elicitation & Analysis Process



Requirement Elicitation & Analysis Process

- **Requirements Discovery and Understanding:**
  - Interact with stakeholders to gather information through interviews, workshops, surveys, etc.
  - Identify domain requirements and constraints.

- **Requirements Classification and Organization:**
  - Group related requirements and organize them into a coherent structure.
  - Use techniques like viewpoints (grouping requirements by stakeholder) or associating requirements with system subsystems.

Requirement Elicitation & Analysis Process

➢Requirements Prioritization and Negotiation:

- ➢ Identify and resolve conflicts among stakeholder requirements.
- ➢ Prioritize requirements based on factors like importance, urgency, and feasibility.
- ➢ Facilitate stakeholder meetings to reach consensus.

➢Requirements Documentation:

- ➢ Document requirements clearly, concisely, and unambiguously.
- ➢ Use simple language and diagrams to enhance understanding.
- ➢ Utilize shared tools like wikis or cloud-based documents for easy access and collaboration.

Requirement Elicitation Techniques & Tools

- **Interview**
  - **Purpose:** To understand stakeholder needs and how they might use a new system.
  - Approaches:
    - **Interviews:**
      - Closed: Predefined set of questions.
      - Open: Exploratory, with no fixed agenda.
    - **Observation/Ethnography:** Observing stakeholders in their work environment.

Requirement Elicitation Techniques & Tools

- **Challenges of Interviewing:**
  - **Stakeholder Uncertainty**: May not clearly articulate needs or may make unrealistic demands.
  - **Communication Gap**: Jargon, implicit knowledge, and differing perspectives can hinder understanding.
  - **Political Factors:** Organizational politics can influence requirements.

- **Effective Interviewing Techniques:**
  - **Open-mindedness**: Be receptive to unexpected requirements.
  - **Active Listening**: Encourage stakeholder input and avoid imposing preconceived notions.
  - **Use of Prompts**: Guide the conversation with questions or scenarios to facilitate discussion.

Requirement Elicitation Techniques & Tools

- **Limitations of Interviews:**
  - May not capture all requirements, especially those that are implicit or difficult to articulate.
  - May not reveal organizational constraints or political influences.

Requirement Elicitation Techniques & Tools

- Ethnography
  - **Observational Technique:** Involves immersing oneself in the work environment of stakeholders.
  - **Focus:** Understanding the actual work practices, social interactions, and organizational context.
  - **Value:**
    - Discovers implicit requirements that may not be explicitly stated.
    - Reveals how people work, not just how they are supposed to work.
    - Provides insights into the social and organizational factors that influence work.

Requirement Elicitation Techniques & Tools

- **Benefits of Ethnography:**

  - **Uncovers Hidden Requirements:**
    - Identifies workarounds and informal practices that may not be documented.
    - Reveals requirements related to cooperation and awareness of other people's activities.

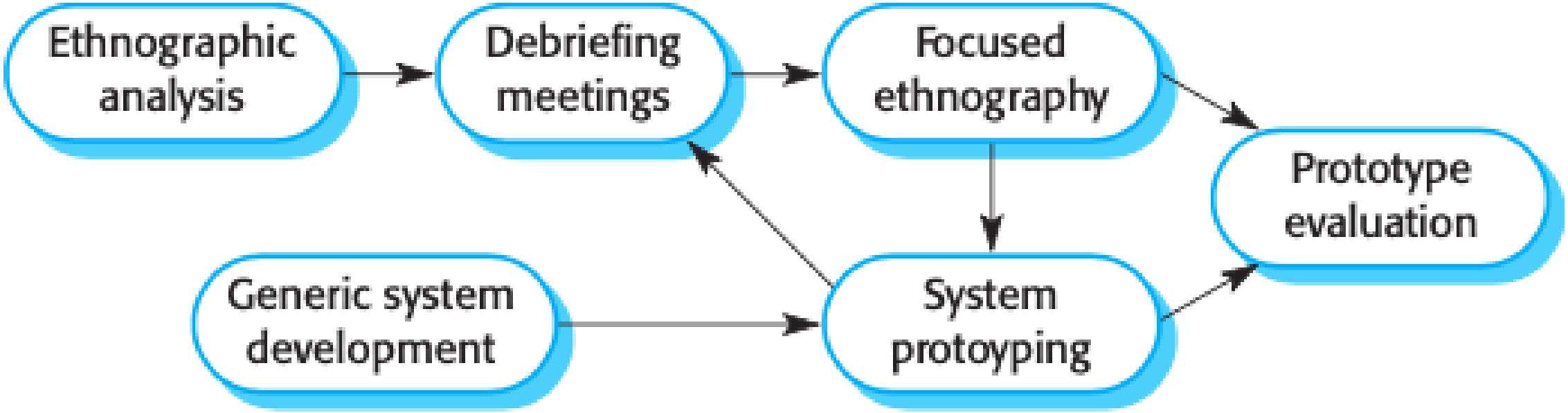
  - **Improves System Design:**
    - Informs prototype development, leading to more user-centered designs.
    - Reduces the number of prototype refinement cycles.

Requirement Elicitation Techniques & Tools

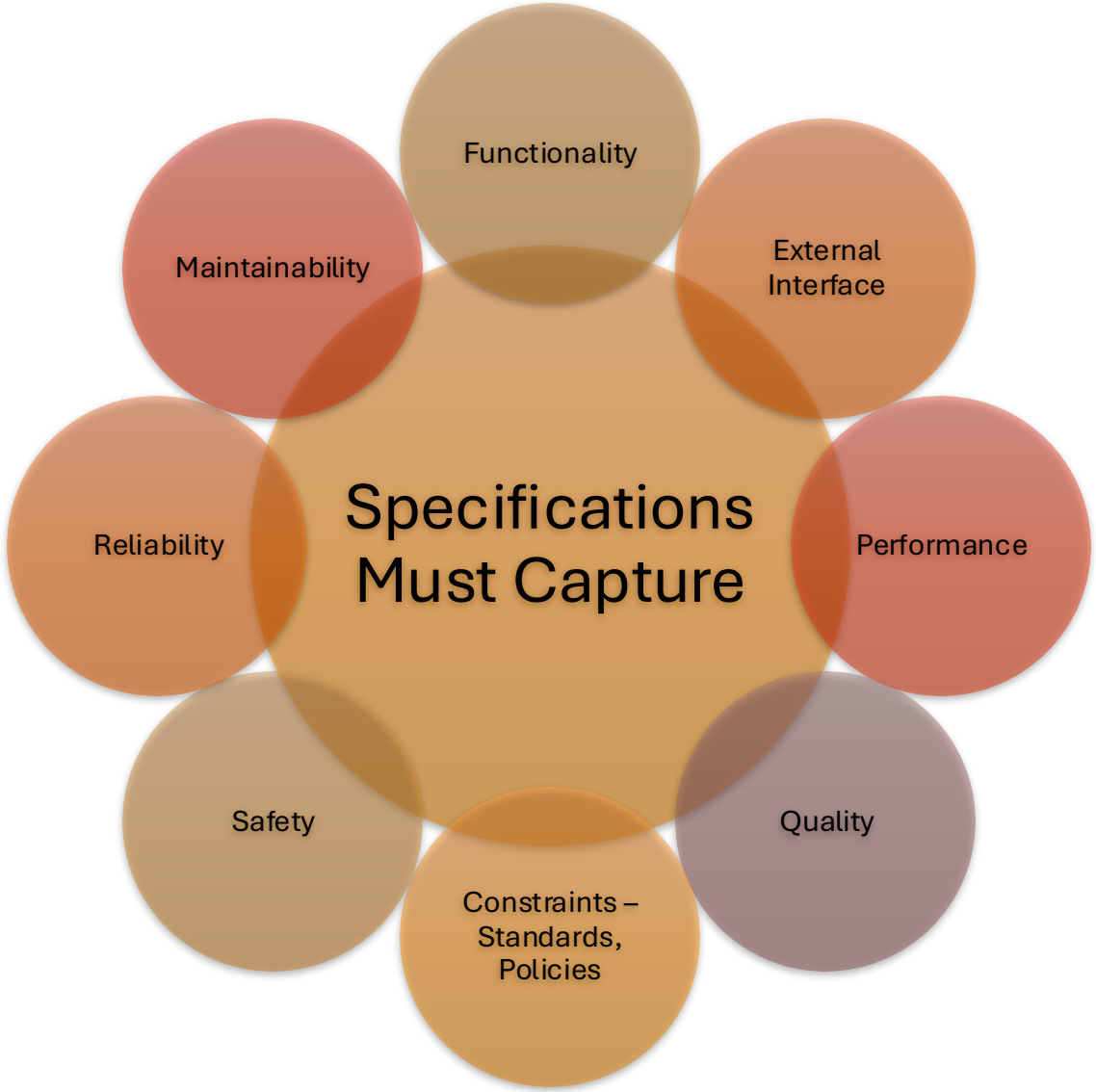
- **Limitations of Ethnography:**

  - **Focus on End-User Perspective:** May not effectively capture broader organizational or domain requirements.
  - **Limited for Innovation:** May not be the best approach for groundbreaking, innovative system designs.
  - **Time-Consuming:** Can be time-intensive and resource-intensive to conduct.

Requirement Elicitation Techniques & Tools



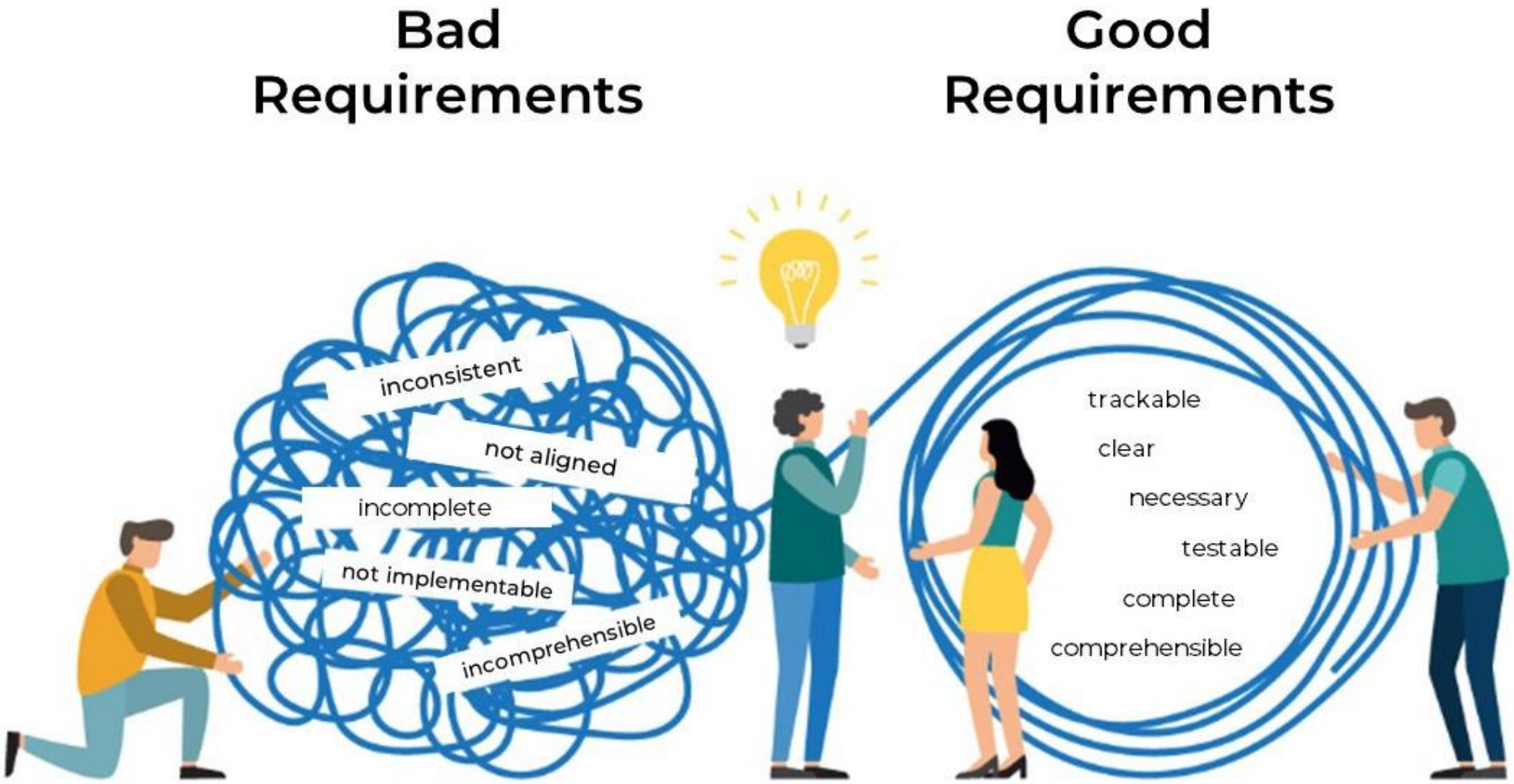
Requirement Specifications



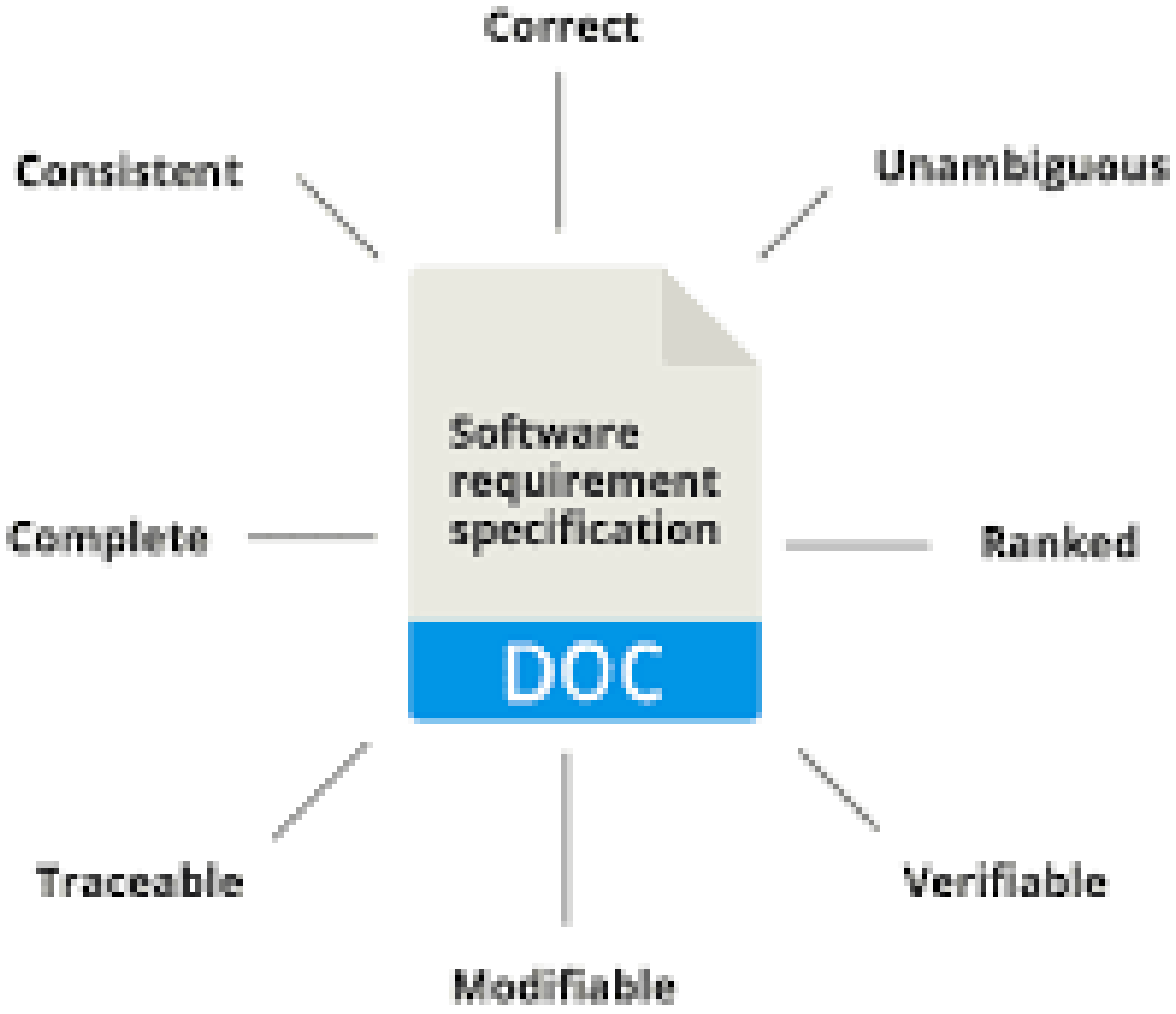
Requirement Specifications – It Captures

- **Functionality**. What is the software supposed to do?
- **External interfaces**. How does the software interact with people, the system's hardware, other hardware, and other software?
- **Performance**. What is the speed, availability, response time, recovery time of various software functions, and so on?
- **Attributes**. What are the portability, correctness, maintainability, security, and other considerations?
- **Design constraints imposed on an implementation**. Are there any required standards in effect, implementation language, policies for database integrity, resource limits, operating environment(s) and so on?

Requirement Specifications – How it should be?



Software Requirement Specifications Characteristics



Software Requirement Specifications Characteristics

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Software Requirement Specifications Characteristics

Table of Contents of an SRS

1. Introduction
   1.1 Purpose
   1.2 Scope
   1.3 Definitions, acronyms, and abbreviations
   1.4 References
   1.5 Overview

2. Overall description
   2.1 Product perspective
   2.2 Product functions
   2.3 User characteristics
   2.4 Constraints
   2.5 Assumptions and dependencies

3. Specific requirements *(the largest & most important part of an SRS, organized in different manner)*

Supporting informations
- Table of contents and Index
- Appendixes *(e.g. Samples I/O formats, background, description of problem to be solved by the software)*

Requirement Specifications – How it should be?

- **Valid (or "correct")**
  - Expresses only the real needs of the stakeholders (customers, users,...)
  - Doesn't contain anything that isn't "required"
- **Unambiguous**
  - Every statement can be read in exactly one way
- **Complete**
  - Specifies all the things the system must do and all the things it must not do!
- **Conceptual Completeness**
  - E.g. responses to all classes of input !Structural Completeness
  - E.g. no TBDs!!!
- **Understandable (Clear)**
  - E.g. by non-computer specialists

Requirement Specifications – How it should be?

- **Consistent**
  - Doesn't contradict itself " I.e., is satisfiable
  - Uses all terms consistently
- **Ranked**
  - Must indicate the importance and/or stability of each requirement
- **Verifiable**
  - A process exists to test satisfaction of each requirement
  - every requirement is specified behaviorally"
- **Modifiable**
  - Can be changed without difficulty
  - Good structure and cross-referencing
- **Traceable**
  - Origin of each requirement must be clear
  - Facilitates referencing of requirements in future documentation

Requirement Management in Agile

➢ Stories and Scenarios

- **Purpose:** To capture real-world usage scenarios and elicit system requirements.
- **Stories:**
  - High-level narratives describing how the system might be used.
  - Effective for setting the "big picture" and facilitating broad discussions.
  - Can be used to gather input from a wider audience (e.g., through wikis).
- **Scenarios:**
  - More detailed descriptions of specific user interactions.
  - Typically structured with information about inputs, outputs, normal flow, error handling, and system state.
  - Can be used to derive specific system requirements.

Requirement Management in Agile

- **Relationship between Stories and Scenarios:**
  - Stories provide the overall context, while scenarios delve into specific use cases.
  - Stories can be used to generate more detailed scenarios.

- **Example:**
  - **Story:** A teacher uses the iLearn system to support student projects on the fishing industry.
  - **Scenario:** A student uploads photos to the KidsTakePics system. The teacher moderates the photos before they are shared.

Requirement Management in Agile

- **Benefits of Using Stories and Scenarios:**
  - **Improved Communication:** Facilitates discussions and helps stakeholders understand the system in a more concrete way.
  - **Real-World Focus:** Helps ensure that the system meets the actual needs of users in real-world situations.
  - **Early Identification of Issues:** Allows for early identification and resolution of potential problems.